

アプリ解説書 / Technical Guide

「シン・探せいとかい：II」

校内謎解きスタンプラリー Web アプリ
仕組みとコードの解説

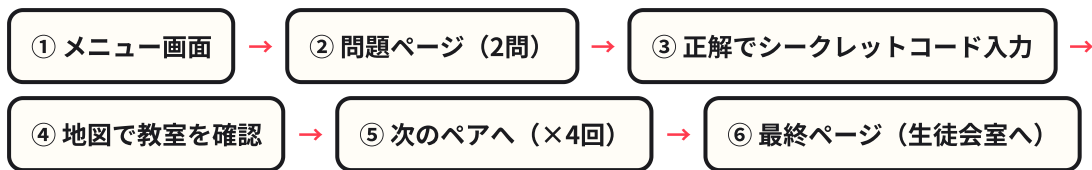
対象ファイル：index.html / プロジェクトファンクション.js /
プロジェクトデザイン.css

このPDFは、アプリの遊び方・全体構成・各コードの役割をまとめたものです。

1. このアプリについて

「シン・探せいとかい：II」は、スマホやPCのブラウザだけで遊べる、校内を使った謎解きスタンプラリーです。生徒は4組の謎（「場所」を当てる問題と「特徴」を当てる問題のペア）を順番に解きながら、実際の教室を巡り、最後に生徒会室にたどり着く……という設計になっています。サーバーやアプリのインストールは不要で、`index.html` をブラウザで開くだけで動作する「フロントエンドだけの1枚もの」のWebアプリです。

1-1. 遊びの流れ



- 全部で4組（8問）。1ページに2問ずつ表示され、1問目に正解すると2問目のロックが外れます。
- 1組を解き終えるたびに「シークレットコード」の入力・QRスキャン画面が挟まり、正しいコードを入れると次のページに進みます。
- 4組すべて終わると、最終ページで宝箱を開けるコードを入力し、クリア演出（紙吹雪＋トロフィー）が表示されます。
- 4組の出題順は毎回ランダムですが、「場所」問題と「特徴」問題のペアは必ず一緒に出ます（`shufflePairsKeepingOrder()` が担当）。

1-2. 主な機能

機能	内容
日英2言語対応	メニューの言語セレクトで日本語⇄英語を切り替え可能。全テキストが <code>translations</code> オブジェクトから読み込まれる。
3種類のテーマ	デフォルト／白黒コミック／ネオンの3テーマ。CSS変数（ <code>--ink</code> など）の値だけを差し替える仕組み。
ヒントシステム	間違えるたびにカウントし、3回でヒント1、5回でヒント2、7回で「4択に変える」救済機能が解放。
校舎見取り図	今見つけた教室をハイライトするSVG製の地図。ピンチ／ホイールでズーム可能。
QRコードスキャン	<code>jsQR</code> ライブラリでカメラ映像からQRコードを読み取り、自動でコード欄に入力。

簡易カンニング対策	右クリック・印刷・PrintScreen・開発者ツールのショートカットをブロックし、注意メッセージを表示。
ページ遷移アニメーション	画面が切り替わるたびにふわっとフェード＋スライドする演出。
背景の漂うアニメーション	星や丸が背景でゆっくり浮遊するデコレーション。
最終ページの案内カード	クリア後に「ここにきてね〜！」と生徒会室へ誘導するカードを表示。

1-3. ファイル構成

index.html	画面のHTML構造（メニュー・問題・コード入力・最終ページの4画面）
プロジェクトファンクション.js	すべてのロジック（約1,900行）。ゲーム進行・判定・地図描画・QR・演出など
プロジェクトデザイン.css	見た目とアニメーション（約2,100行）。テーマ切り替えの仕組みもここ

💡 動かし方

この3つのファイルを同じフォルダに置き、`index.html` をブラウザで開くだけで遊べます。サーバー設置やビルド作業は不要です。スマホのカメラでQRコードを読ませたい場合は、`https://`（もしくは `localhost`）経由で開く必要があります（ブラウザの制約でカメラ権限は `file://` だと動かないことがあります）。

2. 画面構成 (index.html)

`index.html` には4つの「画面」が最初からすべて書かれていて、JavaScript側で `hidden` クラスを付け外しすることで1つずつ切り替えて見せています（1枚のHTMLの中に4画面ぶんのDOMが同居しているイメージです）。

2-1. 4つのscreen

id	役割
<code>menuScreen</code>	タイトル・言語/テーマ選択・ルール説明・スタートボタン
<code>questionScreen</code>	1ページ2問ぶんの謎解きカードを表示するエリア（中身はJSが動的に組み立てる）
<code>intermediateCodeScreen</code>	ペア正解後のシークレットコード入力・QRスキャン・地図表示
<code>codeScreen</code>	最終クリア画面。トロフィー演出と「ここにきてね〜！」カード

仕組みのポイント

すべての画面は `class="screen"` を持ち、表示中でない画面には `hidden`（`display:none`）が付き
ます。画面を切り替える関数 `showScreen(screenName)` が、「今出ている画面をふわっと消す→新しい
画面をふわっと出す」というアニメーション付きの入れ替えを担当しています（詳しくは4章）。

2-2. その他の要素

- `#stampTray`：ペアをクリアするたびにスタンプが増えていく表示欄。
- `#previousAnswerBox`：直前に見つけた教室の答えを表示するボックス（`intermediate`画面内）。
- `#mapContainer`：地図SVGを流し込む場所。ボタンで開閉するフルスクリーンのオーバーレイになる。
- `.bg-decor`：背景でゆっくり漂う星や丸の装飾（今回追加）。純粋にCSSアニメーションだけで動く。

3. ゲームの中身（プロジェクトファンクション.js）

3-1. 問題データ：questions配列

すべての謎解き問題は、ファイル冒頭の `questions` という配列に、1問1オブジェクトとして並んでいます。「場所」問題（`type: "choice"`）と「特徴」問題（`type: "riddle"`）が交互に並び、隣り合う2問が1つのペアになります。

```
{ id: 1, category: "タイプA1", categoryName: "場所", type: "choice",
  grade: 1, classCount: 8, correctClass: 3,
  puzzleImage: ['puzzle_fire', 'puzzle_water', 'puzzle_flask'],
  question: "下のイラストをよく見てみよう...",
  hint1: "ヒント1: ...", hint2: "ヒント2: ..." }
```

プロパティ	意味
<code>type</code>	"choice" = 教室番号を当てる「場所」問題、"riddle" = 言葉を当てる「特徴」 なぞなぞ
<code>grade /</code> <code>correctClass</code>	正解の教室（例：1年3組）。地図のハイライトにもそのまま使われる
<code>puzzleImage</code>	問題文の上に並べるヒントイラスト（アイコンIDの配列）。写真の代わりにSVGアイコンを使うことで、画像ファイルなしで完結させている
<code>answers</code>	「特徴」問題の正解ワード一覧（表記ゆれ・ひらがな/カタカナ違いも複数登録）
<code>hint1 / hint2</code>	段階的に見せるヒント文（英語版は <code>hint1En</code> など）

3-2. ペアの組み立てと出題順：shufflePairsKeepingOrder()

`questions` を2つずつのペアにまとめ、**ペアの順番だけ**をランダムにシャッフルします。ペアの中の2問（場所＋特徴）は分解されないので、「場所」だけ先に全部出る、といったことは起きません。

```
function shufflePairsKeepingOrder(allQuestions) {
  const pairs = [];
  for (let i = 0; i < allQuestions.length; i += 2) {
    pairs.push([ { ...allQuestions[i] }, { ...allQuestions[i + 1] } ]);
  }
  return shuffleArray(pairs).flat();
}
```

各問題を `{ ...q }` でコピーしているのは、教室番号の選択肢（後述の `applyClassroomOptionData`）をプレイ中に毎回シャッフルし直しても、次のプレイに影響が残

らないようにするためです。

3-3. 正誤判定：checkAnswer(slot)

入力欄・4択ボタン・救済4択（ヘルプカード）のどれで答えても、最終的にこの1つの関数がまとめて判定します。処理の流れは次のとおりです。

1. 入力形式（テキスト入力／ラジオボタン）に応じて回答値を取得
2. `normalizeAnswer()` で全角半角・記号・大文字小文字などのゆれを吸収してから比較
3. 正解なら：結果表示・地図に「見つけた！」フラッシュ表示・1問目なら2問目を解禁、2問目ならスタンプ付与＋次の画面へ
4. 不正解なら：同じ間違いを2回言っていないかチェックしつつ、間違い回数をカウントしてヒント解放条件を更新

```
const normalized = normalizeAnswer(userAnswer);
const accepted = getAcceptedAnswers(question);
const correct = accepted.some(answer => normalizeAnswer(answer) === normalized);
```

3-4. ヒント・救済機能

間違い回数	解放される機能
3回	ヒント1（ <code>showHint1</code> 、1問につき2回まで見られる）
5回	ヒント2（ <code>showHint2</code> ）
7回	「4択に変える」ヘルプカード（ <code>activateHelpCard</code> / <code>activateEasyMode</code> ）。ゲーム中1回だけ使える救済措置

3-5. 校舎の地図（SVG描画）

写真素材を使わず、教室の並びを丸ごとJavaScriptでSVGとして描画しています。中心になるのは `buildFloorPlanSvg(question)` です。

- `FLOOR_INFO` / `FLOOR_ORDER`：4F・3F・2Fの3フロアぶんの見出し情報
- `buildFloorBand()`：1フロアぶんの教室の並び（四角いマス）を生成。正解の教室だけ緑色にして「見つけた！」の吹き出しピンを立てる
- `buildFloorPlanSvg()`：3フロアぶんの `buildFloorBand()` 結果と、屋根・体育館・方位磁針（コンパス）などの装飾をまとめて1枚のSVGにする

- `getPreviousMapQuestion()` : 地図に出していいのは「直前に解いたペアの教室」だけ、というネタバレ防止ルールを担当
- `attachMapZoomPan()` : マウスホイール／ピンチ／ドラッグでの拡大縮小・パン操作を実装

※ 今回修正したバグ：地図の謎の丸ハイライト

地図右上にあるコンパス（方位磁針）バッジと屋根の頂点が、地図カードの角ギリギリ（丸められた角のすぐ内側）に配置されていました。フルスクリーン表示のとき地図は「高さいっぱい引き伸ばす」処理をしていたため、横幅が枠からわずかにみ出し、`overflow:hidden` と角丸によってコンパスの一部が欠けて見えていたのが、スクリーンショットに写っていた「謎の丸」の正体でした。今回、①コンパスと屋根の頂点をカードの端から余白を取ってずらす、②地図SVGを「枠の中に収まるまで縮小する」方式（`width:100%; height:100% + SVGのpreserveAspectRatio="xMidYMid meet"`）に変更、③地図の枠自体を地図の縦横比に合わせた固定の長方形にする、という3つの修正で、どんな画面サイズでも欠けずにきれいな長方形の中に収まるようにしました。

3-6. 多言語対応：translations

画面に出る文字列はほぼすべて、`translations.ja / translations.en` という2つのオブジェクトに集約されています。`applyLanguageUI()` という1つの関数が、現在の `gameState.language` を見て、画面上の各要素のテキストを一括で書き換えます。新しい文言を増やしたいときは、**この2つのオブジェクトに同じキー名で日本語版・英語版を追加する**のが基本の作法です。

3-7. テーマ切り替え

テーマは「色や背景をCSS変数だけで丸ごと差し替える」という仕組みです。`applyTheme(theme)` が `<html data-theme="mono">` のように属性を付け替え、`プロジェクトデザイン.css` 側の `html[data-theme="mono"] { --ink: ...; --paper: ...; }` のようなルールが変数の値を上書きします。ボタンやカードなど個々の部品は全部 `var(--ink)` のようにCSS変数経由で色を参照しているため、新しいテーマを増やしたいときも、このプロパティのセットを1つ追加するだけで済みます。テーマ切り替え時の泡が広がるようなトランジション演出は `runThemeBubbleTransition()` が担当しています。

3-8. 演出・アニメーション系の関数

関数	役割
<code>celebrateCorrectAnswer()</code>	正解時の紙吹雪（confetti）を画面上に降らせる
<code>flashRoomConfirmation()</code>	「場所」問題正解時、見つけた教室を地図付きでポップアップ表示

<code>addStamp()</code>	ペアクリアごとにスタンプトレイへスタンプを1つ追加
<code>showScreen()</code>	画面切り替え時のフェード+スライドアニメーション（4章で詳説）
<code>renderFinalCongratsContent()</code>	最終ページのトロフィー演出+「ここにきてね〜！」案内カードを組み立てる

3-9. QRコードスキャン

`jsQR`（CDN経由で読み込み）を使い、カメラ映像を `<canvas>` に1フレームずつ描画してQRコードを解析しています。`toggleIntermediateQRScanner()` がカメラ起動、`requestAnimationFrame` ループで毎フレームスキャンし、コードを検出したら自動で入力欄に反映して `submitIntermediateCode()` を呼び出します。

3-10. 簡易カンニング対策

ファイル冒頭で、右クリック・`Ctrl/Cmd+P`（印刷）・`PrintScreen`・`F12` や `Ctrl+Shift+I`（開発者ツール）をそれぞれイベントリスナーでブロックし、注意メッセージを表示します。

⚠ 注意

これはあくまで「気軽な悪ふざけ防止」程度の簡易的な対策です。ブラウザの開発者ツールを完全に禁止する方法は存在しないため、本気で防ぐための仕組みではなく、雰囲気づくり・軽い抑止のためのものと理解しておいてください。

4. 今回追加・修正した機能

4-1. ページ遷移アニメーション

画面の切り替えはすべて `showScreen(screenName)` という1つの関数を通ります。今回、この関数を「今の画面にフェードアウト（`screen-leave`）→260ms後に非表示にして次の画面をフェードイン（`screen-enter`）」という2段階の処理に書き換えました。

```
const SCREEN_TRANSITION_MS = 260;

function showScreen(screenName) {
  const target = document.getElementById(screenName);
  const current = SCREEN_IDS.map(id => document.getElementById(id))
    .find(el => el && el !== target && !el.classList.contains('hidden'));

  const activateTarget = () => {
    // 他の画面を隠し、target に screen-enter を付けてフェードイン
  };

  if (!current) { activateTarget(); return; }
  current.classList.add('screen-leave');
  setTimeout(activateTarget, SCREEN_TRANSITION_MS);
}
```

対応するCSS側は、不透明度と `translateY`・`scale` をトランジションさせるだけのシンプルな仕組みです。`prefers-reduced-motion: reduce`（OS側の「動きを減らす」設定）が有効な場合は、アニメーションなしで即座に切り替わるようにしています。

また、1ページに並ぶ2問の問題カード自体にも `cardPopIn` というポップインアニメーションを追加し、2問目のカードは1問目より少し遅れて出てくる（`animation-delay`）ようにして、ページが切り替わるたびに気持ちの良いテンポが生まれるようにしています。

4-2. 背景の漂うアニメーション

`index.html` の `<body>` 直下に、星や丸の記号を並べた `<div class="bg-decor">` を追加しました。それぞれの図形は `position: fixed` で背面（`z-index: 0`）に固定し、`bgShapeFloat` というキーフレームアニメーションでゆっくり上下に漂う＋回転するようにしています。ゲーム本体のカードは `z-index: 1` を指定して常に手前に表示されるようにしてあるので、操作の邪魔にはなりません。ここも `prefers-reduced-motion` 設定を尊重します。

4-3. 地図バグ修正＋長方形コンテナ化

3-5節で説明したとおり、コンパスと屋根の頂点の位置を余白付きで調整し、地図SVGの表示方式を「枠の中に収まるまで縮小する（`preserveAspectRatio="xMidYMid meet"`）」方式に変更しました。さらに地図の外枠（`.game-map-viewport`）に、地図の実際の縦横比（400:378）に合わせた`aspect-ratio`を指定し、画面サイズが変わっても常にきれいな長方形の枠の中にぴったり収まるようにしています。

4-4. 最終ページ：「ここにきてね〜！」案内カード

全問クリア後のトロフィー演出のすぐ下に、生徒会室へ誘導する新しいカードを追加しました。今はまだ実際の写真がないため、ドアとネームプレート（「生徒会室」）を模したイラストをその場で仮表示しています（`gameSvgIcons.final_invite_door`、SVGなので画像ファイル不要）。

実際の写真に差し替えるには

プロジェクトファンクション.js 内の `renderFinalCongratsContent()` にある `${getGameIconSvg('final_invite_door')}` の行を、次のように書き換えるだけです。

```

```

写真ファイル（生徒会室.jpg など好きなファイル名でOK）を `index.html` と同じフォルダに置けば、そのまま表示されます。枠のデザイン（`.final-invite-photo-frame`）はそのまま使われるので、差し替えるだけで見た目が整った状態になります。

文言は `translations.ja.finalInviteHeading` / `finalInviteSub`（英語版は `translations.en` 側）で管理しているので、言葉を変えたいときはそこを編集してください。

5. よく使うカスタマイズ早見表

やりたいこと	編集する場所
問題文・正解・ヒントを変える	プロジェクトファンクション.js の <code>questions</code> 配列
シークレットコードを変える	<code>SHARED_SECRET_CODE</code> （配列内のどれか1つが一致すればOK）
教室の並び・階数表示を変える	<code>FLOOR_INFO</code> / <code>FLOOR_ORDER</code>
日本語・英語の文言を変える	<code>translations.ja</code> / <code>translations.en</code>
色・テーマを変える／増やす	プロジェクトデザイン.css の <code>:root</code> と <code>html[data-theme="..."]</code>
最終ページの写真を差し替える	<code>renderFinalCongratsContent()</code> （4-4節を参照）
ヒント解放の間違い回数を変える	<code>checkAnswer()</code> 内の間違い回数の判定部分（3回・5回・7回の箇所）
背景の漂う飾りを増減する	<code>index.html</code> の <code>.bg-decor</code> 内の <code></code> を追加・削除

動作確認のヒント

ブラウザで `index.html` を開いた状態で右クリック→「検証」（開発者ツール）を使いたい場合は、3-10節の簡易カンニング対策が邪魔になります。編集集中だけ、ファイル冒頭の
`document.addEventListener('contextmenu', ...)` 等のブロックを一時的にコメントアウトすると作業しやすくなります。

このPDFは、アップロードされた `index.html`／`プロジェクトファンクション.js`／`プロジェクトデザイン.css` の内容をもとに、Claude が自動生成した解説資料です。コードは今後の編集で変わる可能性があるため、最新の挙動は実際のファイルを優先してください。